**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Национальный исследовательский университет ИТМО»**

**Факультет Программной Инженерии и Компьютерной Техники**

# Блок задач с Timus

Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа P3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

# Содержание

# 1160. Network

## Условие:

Andrew is working as system administrator and is planning to establish a new network in his company. There will be N hubs in the company, they can be connected to each other using cables. Since each worker of the company must have access to the whole network, each hub must be accessible by cables from any other hub (with possibly some intermediate hubs).

Since cables of different types are available and shorter ones are cheaper, it is necessary to make such a plan of hub connection, that the maximum length of a single cable is minimal. There is another problem - not each hub can be connected to any other one because of compatibility problems and building geometry limitations. Of course, Andrew will provide you all necessary information about possible hub connections.

You are to help Andrew to find the way to connect hubs so that all above conditions are satisfied.

### Исходные данные

The first line contains two integer: N - the number of hubs in the network ($2 \le N \le 1000$) and M — the number of possible hub connections ($1 \le M \le 15000$). All hubs are numbered from 1 to N. The following M lines contain information about possible connections - the numbers of two hubs, which can be connected and the cable length required to connect them. Length is a positive integer number that does not exceed $10^6$. There will be no more than one way to connect two hubs. A hub cannot be connected to itself. There will always be at least one way to connect all hubs.

### Результат

Output first the maximum length of a single cable in your hub connection plan (the value you should minimize). Then output your plan: first output P - the number of cables used, then output P pairs of integer numbers - numbers of hubs connected by the corresponding cable. Separate numbers by spaces and/or line breaks.

### Пример

| исходные данные | результат |
|---|---|
| 4 6<br>1 2 1<br>1 3 1<br>1 4 2<br>2 3 1<br>3 4 1<br>2 4 1 | 1<br>4<br>1 2<br>1 3<br>2 3<br>3 4 |

## Решение:

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

struct DSU {
  std::vector<int> parents;

  DSU(int n) {
    parents.resize(n + 1);
    for (int i = 1; i < n + 1; i++) {
      parents[i] = i;
    }
  }

  int find(int node) {
    if (node == parents[node]) {
      return node;
    }

    return parents[node] = find(parents[node]);
  }

  void unite(int node1, int node2) {
    node1 = find(node1);
    node2 = find(node2);

    if (node1 != node2) {
      parents[node2] = node1;
    }
  }
};

int main() {
  int hubs_count, possible_hub_connections_count;
  std::cin >> hubs_count >> possible_hub_connections_count;

  std::vector<std::pair<std::pair<int, int>, int>> hubs;
  for (int i = 0; i < possible_hub_connections_count; i++) {
    int hub1, hub2, dist;
    std::cin >> hub1 >> hub2 >> dist;

    hubs.push_back({
        {hub1, hub2},
        dist
    });
  }

  std::sort(hubs.begin(), hubs.end(), [](const auto& a, const auto& b) {
    return a.second < b.second;
  });
```

```
  int max_dist = 0;
  DSU dsu(hubs_count);
  std::vector<std::pair<int, int>> results;

  for (auto& h : hubs) {
    int first_hub = h.first.first;
    int second_hub = h.first.second;

    if (dsu.find(first_hub) != dsu.find(second_hub)) {
      dsu.unite(first_hub, second_hub);
      results.push_back({first_hub, second_hub});
      max_dist = std::max(max_dist, h.second);
    }
  }

  std::cout << max_dist << std::endl;
  std::cout << results.size() << std::endl;
  for (auto& r : results) {
    std::cout << r.first << " " << r.second << std::endl;
  }
}
```

## О решении:

Идея:
Условие нахождение минимального кабеля (который является максимальным среди всех) эквивалентно требованию построения минимального остовного дерева, который соединяет все вершины без циклов (аналогично условию соединения всех хабов) и имеет минимальную сумму ребер (в таком случае максимальное ребро будет минимально). Используем алгоритм Крускала. Сортируем пары хабов по длине соединительного кабеля между ними, добавляем их по возрастанию – если хабы были в разных ветвях, то склеиваем их. Продолжаем, пока не соединим все. Таким образом, все хабы будут в одной группе, можно добрать из любого в любой, группа содержит каждый хаб ровно по разу.

Сложность по времени:
O(n log n) – так как сортируем массив пар хабов по длине кабеля.

Сложность по памяти:
O(n + m) – так как храним DSU и пары хабов.

# 1162. Currency Exchange

## Условие:

Several currency exchange points are working in our city. Let us suppose that each point specializes in two particular currencies and performs exchange operations only with these currencies. There can be several points specializing in the same pair of currencies. Each point has its own exchange rates, exchange rate of A to B is the quantity of B you get for 1A. Also each exchange point has some commission, the sum you have to pay for your exchange operation. Commission is always collected in source currency.

For example, if you want to exchange 100 US Dollars into Russian Rubles at the exchange point, where the exchange rate is 29.75, and the commission is 0.39 you will get (100 - 0.39) * 29.75 = 2963.3975RUR.

You surely know that there are N different currencies you can deal with in our city. Let us assign unique integer number from 1 to N to each currency. Then each exchange point can be described with 6 numbers: integer A and B - numbers of currencies it exchanges, and real RAB, CAB, RBA and CBA - exchange rates and commissions when exchanging A to B and B to A respectively.

Nick has some money in currency S and wonders if he can somehow, after some exchange operations, increase his capital. Of course, he wants to have his money in currency S in the end. Help him to answer this difficult question. Nick must always have non-negative sum of money while making his operations.

### Исходные данные

The first line contains four numbers: N - the number of currencies, M - the number of exchange points, S - the number of currency Nick has and V - the quantity of currency units he has. The following M lines contain 6 numbers each - the description of the corresponding exchange point - in specified above order. Numbers are separated by one or more spaces. $1 \le S \le N \le 100$, $1 \le M \le 100$, V is real number, $0 \le V \le 10^3$.

For each point exchange rates and commissions are real, given with at most two digits after the decimal point, $10^{-2} \le \text{rate} \le 10^2$, $0 \le \text{commission} \le 10^2$.

Let us call some sequence of the exchange operations simple if no exchange point is used more than once in this sequence. You may assume that ratio of the numeric values of the sums at the end and at the beginning of any simple sequence of the exchange operations will be less than $10^4$.

### Результат

If Nick can increase his wealth, output YES, in other case output NO.

### Примеры

| исходные данные | результат |
|---|---|
| 3 2 1 10.0<br>1 2 1.0 1.0 1.0 1.0<br>2 3 1.1 1.0 1.1 1.0 | NO |
| 3 2 1 20.0<br>1 2 1.0 1.0 1.0 1.0<br>2 3 1.1 1.0 1.1 1.0 | YES |

## Решение:

```
#include <iostream>
#include <vector>

struct exchange_info {
  int from;
  int to;
  double rate;
  double comission;
};

int main() {
  int currencies_count, exchange_points_count;
  int start_currency_number;
  double start_currency_amount;

  std::cin >> currencies_count >> exchange_points_count;
  std::cin >> start_currency_number >> start_currency_amount;

  std::vector<exchange_info> graph;
  for (int i = 0; i < exchange_points_count; i++) {
    int currency_a, currency_b;
    double rate_a_to_b, comission_a_to_b;
    double rate_b_to_a, comission_b_to_a;

    std::cin >> currency_a >> currency_b;
    std::cin >> rate_a_to_b >> comission_a_to_b;
    std::cin >> rate_b_to_a >> comission_b_to_a;

    graph.push_back({currency_a, currency_b, rate_a_to_b, comission_a_to_b});
    graph.push_back({currency_b, currency_a, rate_b_to_a, comission_b_to_a});
  }

  std::vector<double> money(currencies_count + 1, 0.0);
  money[start_currency_number] = start_currency_amount;

  for (int i = 0; i < currencies_count - 1; i++) {
    for (auto& g : graph) {
      if (money[g.from] > g.comission) {
        double new_money = (money[g.from] - g.comission) * g.rate;

        if (new_money > money[g.to]) {
          money[g.to] = new_money;
        }
      }
    }
  }

  bool can_become_dollar_multimillionaire = false;

  for (auto& g : graph) {
    if (money[g.from] > g.comission) {
```

```
      double new_money = (money[g.from] - g.comission) * g.rate;

      if (new_money > money[g.to]) {
        can_become_dollar_multimillionaire = true;
        break;
      }
    }
  }

  if (can_become_dollar_multimillionaire) {
    std::cout << "YES";
  } else {
    std::cout << "NO";
  }
}
```

## О решении:

Идея:
Задача сводится к поиску положительного цикла в графе обменов. Вершины графа – валюты, а направленные ребра в обе стороны – формулы обмена из одной валюты в другую. Выполняем N-1 итерацию для вычисления новых значений после первого простого пути (так как это максимальная длина простой цепочки из N вершин), на каждом шаге пытаемся увеличить стоимости. Если на N проходе удается хоть где-то получить сумму больше исходной, то найти арбитражное окно можно. Иначе нет.

Сложность по времени:
O(n * m) – n итераций (валюты) по m ребрам (конверсии валют).

Сложность по памяти:
O(n + m) – так как храним массив денежных значений размера количества валют и все пары обменников.